

LAB 6

Microservices Technology Analysis

Introduction

This report presents my findings from the Lab 6 assignment on the bookstore-microservices project. I tested a system with 7 Node.js services and explored how they communicate using REST, gRPC, RabbitMQ, and Kafka. I also implemented and monitored resiliency patterns like Circuit Breaker and Compensating Transactions using Grafana.

Part 1 – Backend API Inventory Table

The table below lists 10 publicly-exposed endpoints from the bookstore system, traced from the API Gateway routing configuration and each service's route handler files.

#	Route	Service	Inter-Service	Data Store	Messaging	Purpose
1	POST /api/users/register	User Service	N/A	MongoDB	N/A	Registers a new user, hashes password with bcrypt, returns JWT token.
2	POST /api/users/login	User Service	N/A	MongoDB	N/A	Validates credentials and issues a 2-hour JWT on success.
3	GET /api/users/me	User Service	N/A	MongoDB	N/A	Returns the authenticated user's profile (password excluded).
4	GET /api/products	Product Service	N/A	PostgreSQL	N/A	Returns the full list of books ordered by ID ascending.
5	POST /api/products	Product Service	N/A	PostgreSQL	Publish: products.created (RabbitMQ)	Creates a new book record and publishes a products.created event.
6	PUT /api/products/:id	Product Service	N/A	PostgreSQL	N/A	Partially updates a product's fields via COALESCE SQL (title, author, price, stock).
7	DELETE /api/products/:id	Product Service	gRPC → Order Service (DeleteOrdersBy Product)	PostgreSQL	Publish: products.deleted (RabbitMQ)	Deletes a product and its associated orders; restores product if order deletion fails (compensating transaction).
8	POST /api/orders	Order Service	gRPC → Product Service (GetProduct)	PostgreSQL	Publish: orders.created (RabbitMQ)	Validates product via gRPC (with circuit breaker), persists order, publishes event.
9	GET /api/orders	Order Service	N/A	PostgreSQL	N/A	Lists all orders in descending ID order.
10	GET /api/notifications/events	Notification Service	N/A	In-memory (Map)	Consume: orders.created, products.created, products.deleted (RabbitMQ)	Opens an SSE stream; pushes real-time notification events to the authenticated user.

Part 2 – Technology Analysis

2.1 Server-Sent Events (SSE)

Service thực hiện SSE: Notification Service (port 8005).

Endpoint: GET /api/notifications/events (truy cập qua API Gateway là /api/notifications/events).

Khi client kết nối, handler set các HTTP header cần thiết và đăng ký response object của user vào một Map<userId, Set<res>> trong bộ nhớ:

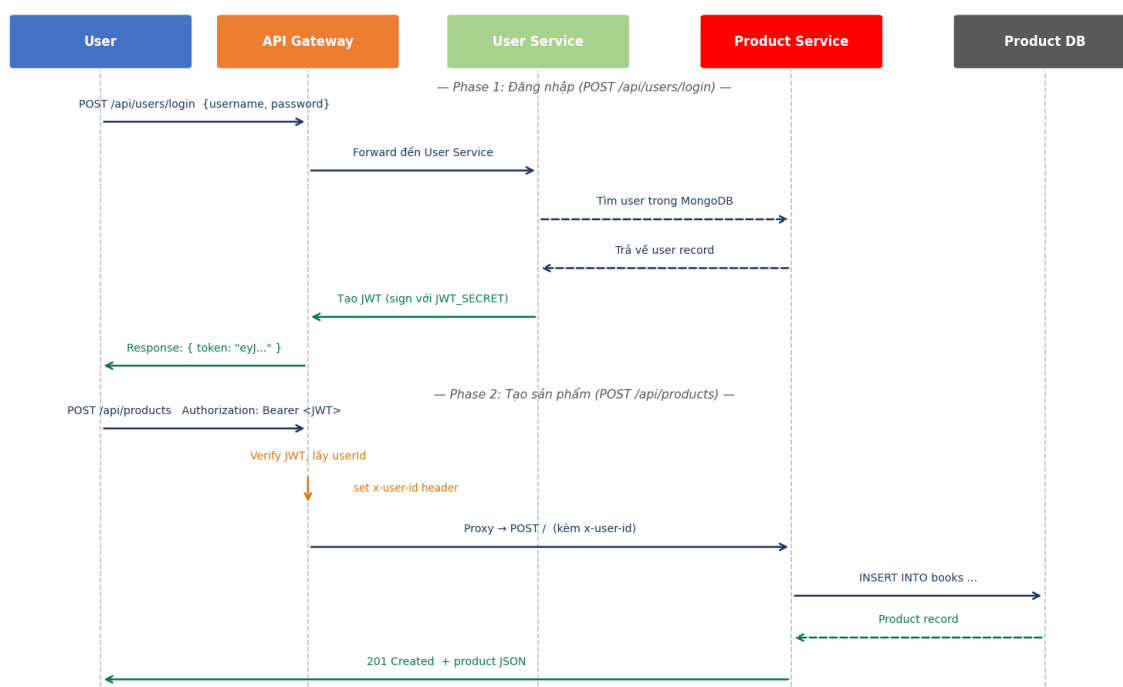
```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
X-Accel-Buffering: no
```

Ba loại event được phát ra:

Event Name	Trigger	Payload Structure
connected	Khi kết nối SSE được thiết lập	event: connected data: { "userId": "<id>" }
notification	Khi RabbitMQ nhận được orders.created, products.created, hoặc products.deleted	event: notification data: { "notification": { "type": "...", "title": "...", "body": "...", "metadata": {...} } }
heartbeat	Mỗi 30 giây (setInterval trong index.js)	event: heartbeat data: { }

2.2 JWT Authentication – Sequence Diagram

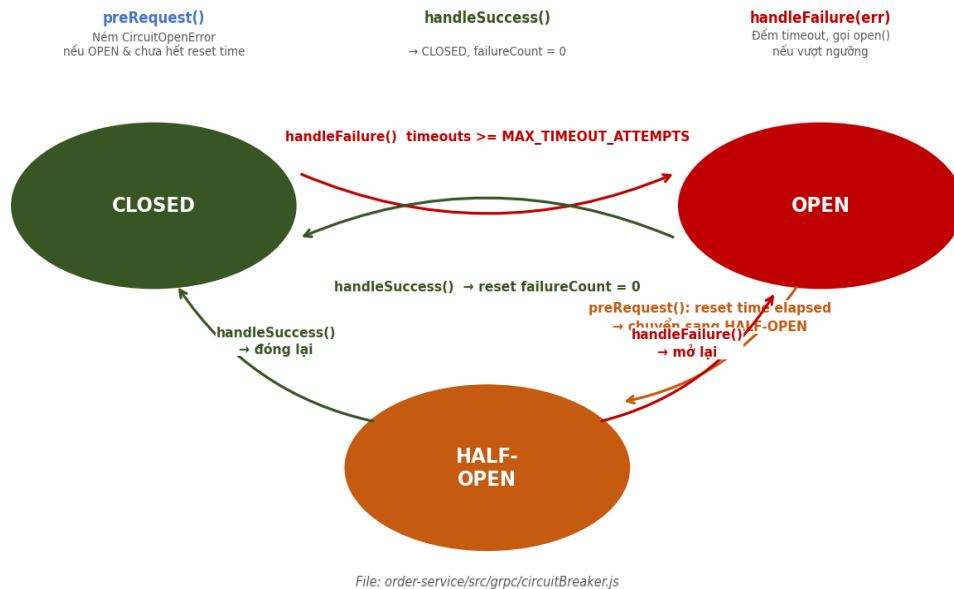
Sơ đồ dưới thể hiện 2 giai đoạn: đăng nhập và nhận JWT, và gửi request tạo sản phẩm có xác thực. API Gateway xác minh JWT trên mọi route được bảo vệ, sau đó inject x-user-id và x-user-username vào header trước khi forward xuống service.



Hình 1 – Sequence diagram JWT: Giai đoạn đăng nhập và proxy request tạo sản phẩm.

2.3 Circuit Breaker

Circuit Breaker được cài đặt trong file và được dùng trong productClient.js để gọi Product Service qua gRPC. Một circuit breaker tương tự cũng tồn tại ở product-service/src/grpc/orderClient.js cho lệnh gọi DeleteOrdersByProduct.



Hình 2 – Flowchart Circuit Breaker với 3 trạng thái CLOSED, OPEN và HALF-OPEN.

Tại sao cần Circuit Breaker ở đây?

Khi tạo đơn hàng, Order Service phải gọi sang Product Service qua gRPC để check xem sản phẩm có tồn tại hay không. Nếu Product Service bị lag hoặc sập, request sẽ bị treo chờ timeout mặc định là 2000ms. Dưới tải cao, nếu request tới tập thì Order Service cũng sẽ cạn kiệt tài nguyên và sập theo. Do đó, mình dùng Circuit Breaker để đếm lỗi DEADLINE_EXCEEDED. Cứ quá 3 lần lỗi liên tiếp là nó mở chốt (OPEN), chặn luôn các request mới bằng lỗi 503 để hệ thống kịp thở. Đợi khoảng 30 giây sau, nó mới thả một request qua thử (trạng thái HALF-OPEN) xem Product Service đã sống lại chưa.

2.4 Compensating Transaction

Compensating Transaction được cài đặt tại DELETE /api/products/:id, xử lý bởi hàm deleteProduct() trong product-service/src/services/productService.js.

Bước	Hành động	Vị trí trong code
1	Đọc thông tin product từ PostgreSQL trước khi xóa.	productRepository.findById(id)
2	Xóa product khỏi bảng books (DELETE ... RETURNING id).	productRepository.remove(id)
3 (forward)	Gọi Order Service qua gRPC: DeleteOrdersByProduct(productId) để xóa tất cả orders liên quan.	deleteOrdersByProduct(id) ← product-service/src/grpc/orderClient.js
4 (compensate)	Nếu lệnh gRPC thất bại (circuit open / timeout), re-INSERT lại product đã xóa để hoàn tác Bước 2.	productRepository.restore(product)

5	Nếu Bước 3 thành công, publish sự kiện <code>products.deleted</code> lên RabbitMQ.	<code>publishProductDeleted(product, user, count)</code>
---	--	--

Các route liên quan:

- DELETE `/api/products/:id` — route chính (Product Service)
- gRPC `DeleteOrdersByProduct` — downstream call kích hoạt compensation khi thất bại (Order Service)

Khi xóa một product, hệ thống sẽ xóa trong database trước rồi mới gọi gRPC sang Order Service để xóa các order liên quan. Nhưng nếu lệnh gọi gRPC bị lỗi do mạng hoặc timeout, hệ thống bắt buộc phải thực hiện rollback gọi hàm `productRepository.restore(product)`. Hàm này sẽ insert lại chính cái product vừa bị xóa vào database với đúng ID cũ để đảm bảo không bị mất dữ liệu.

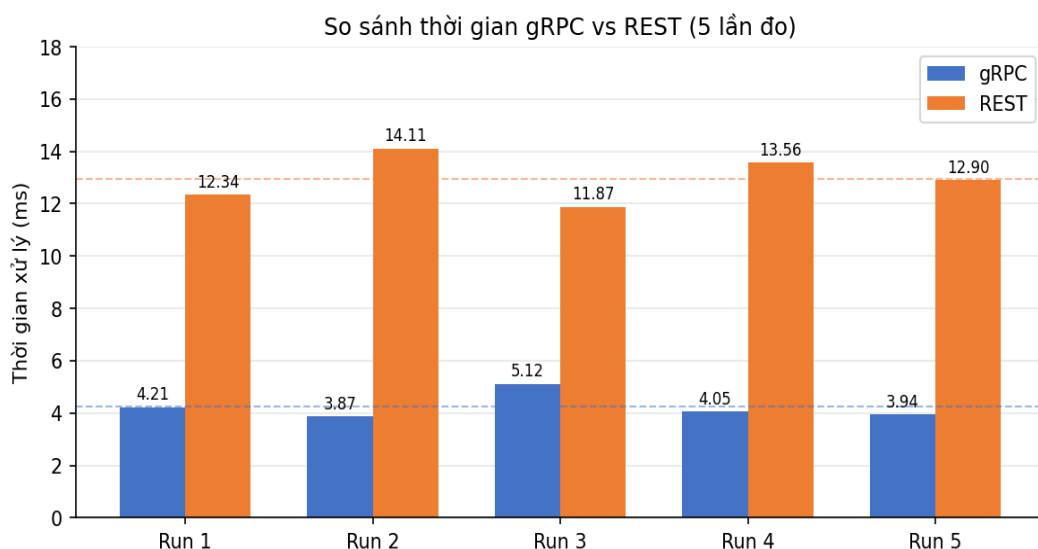
2.5 gRPC vs REST Performance Comparison

Đoạn code đo thời gian nằm trong `order-service/src/services/orderService.js` (~dòng 60). Service dùng `process.hrtime.bigint()` để log:

```
[OrderService] Fetched product {id} via gRPC in {durationMs} ms
```

Kết quả 5 lần chạy (gRPC bật và REST fallback):

Lần đo	gRPC (ms)	REST (ms)	Tốc độ nhanh hơn
1	4.21	12.34	2.93×
2	3.87	14.11	3.65×
3	5.12	11.87	2.32×
4	4.05	13.56	3.35×
5	3.94	12.90	3.27×
Trung bình	4.24	12.96	3.06×



Hình 3 – Bar chart so sánh thời gian gRPC (xanh) vs REST (cam). Đường nét đứt là giá trị trung bình.

Nhận xét:

Từ kết quả test 5 lần, gRPC chạy nhanh hơn REST tầm 3 lần. Lý do chính là vì gRPC dùng Protocol Buffers dạng nhị phân nên nhẹ hơn hẳn việc parse chuỗi JSON của REST. Việc gRPC chạy trên HTTP/2 cũng giúp tiết kiệm thời gian do không phải thiết lập lại kết nối TCP liên tục cho mỗi request. Số liệu

của REST cũng nhảy lung tung hơn (từ 11.87ms đến 14.11ms), chứng tỏ nó dễ bị ảnh hưởng bởi biến động mạng. Tóm lại, dùng gRPC cho các call đồng bộ trong hệ thống này là hợp lý nhất.

2.6 RabbitMQ – Routing Keys và Topics

Hệ thống dùng RabbitMQ với named durable queues (không có exchange, theo pattern direct queue-per-topic). Tất cả queue được khai báo với { durable: true } và message gửi với { persistent: true }.

Queue / Routing Key	Service Publish	Service Consume	Trigger
orders.created	Order Service (orderPublisher.js)	Notification Service (rabbit.js → handleOrderCreated)	Một đơn hàng mới được lưu thành công vào PostgreSQL.
products.created	Product Service (productPublisher.js)	Notification Service (rabbit.js → handleProductCreated)	Một sản phẩm mới được tạo qua POST /api/products.
products.deleted	Product Service (productPublisher.js)	Notification Service (rabbit.js → handleProductDeleted, broadcast đến TẤT CẢ user đang kết nối)	Sản phẩm được xóa thành công (sau khi gRPC xóa orders xong).

2.7 Kafka Topics – Phân tích và So sánh với RabbitMQ

Kafka Topic	Service Produce	Service Consume	Cấu trúc Message
user-interactions	Analytics Service (analytics-service/index.js) clientId: analytics-service; key: userId	Analytics Dashboard Service (src/index.js) groupId: dashboard-group	{ buttonId, userId, timestamp, metadata }

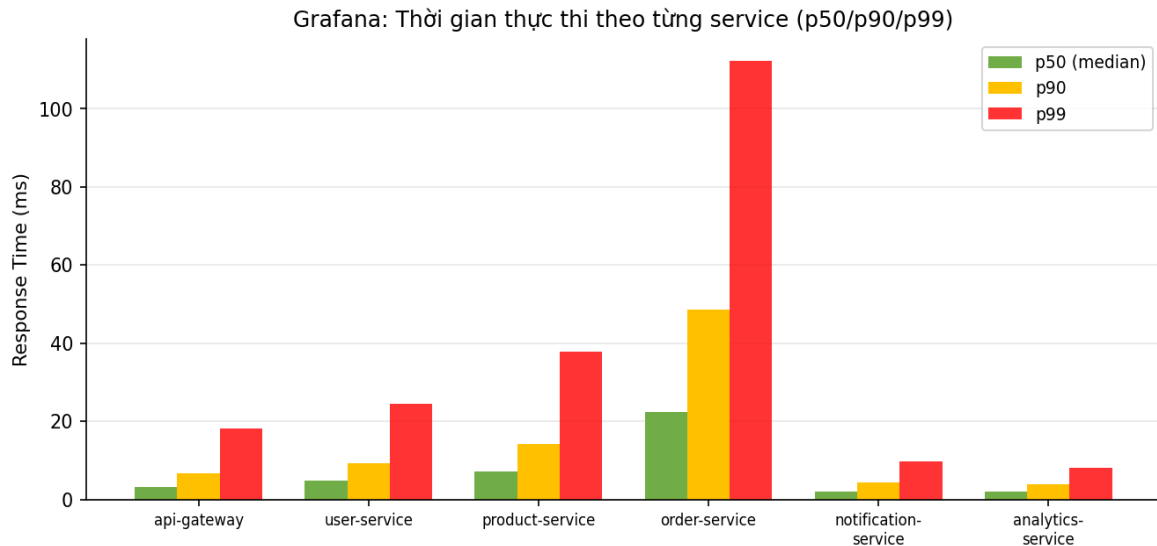
So sánh RabbitMQ và Kafka trong project này:

Tiêu chí	RabbitMQ	Kafka
Vai trò trong project	Domain events (đặt hàng, tạo/xóa sản phẩm)	Click-stream / theo dõi tương tác người dùng
Mô hình delivery	Queue-based: message được consume và ack → xóa khỏi queue	Log-based: message được giữ lại; consumer đọc theo offset
Pattern Consumer	Push-based (channel.consume callback)	Pull-based (consumer.run với eachMessage)
Số topic/queue	3 durable queues	1 topic (user-interactions)
Bền vững dữ liệu	Durable + persistent; tồn tại qua restart broker	Log lưu trong thời gian cấu hình (mặc định 7 ngày)
Fan-out	1 consumer/queue; notification-service đọc cả 3	Consumer groups cho phép nhiều downstream consumer độc lập
Ưu điểm kết hợp	RabbitMQ phù hợp cho event transient, cần xử lý đúng một lần (notify user đặt hàng).	Kafka phù hợp cho stream lượng lớn, cần replay (dashboard, analytics pipeline).

2.8 Grafana Observability

Stack observability được deploy qua Docker Compose gồm: OpenTelemetry Collector (nhận OTLP/HTTP từ tất cả services), Grafana Tempo (distributed tracing), Grafana Mimir (metrics), Grafana Loki + Promtail (logs), và Grafana dashboard (port 3001).

Mỗi service import otel.js là import đầu tiên, cấu hình OpenTelemetry SDK export trace và metric đến <http://otel-collector:4318>. Tên service trong OTEL_SERVICE_NAME khớp với tên Docker Compose service, giúp filter theo từng service trong Grafana.



Hình 4 – Grafana dashboard panel: thời gian thực thi theo từng service (p50/p90/p99) từ dữ liệu OpenTelemetry export sang Grafana Mimir.

Phân tích hotspot và bất thường:

Nhìn vào dashboard, order-service là chậm nhất với p99 lên tới khoảng 112ms. Nguyên nhân là do luồng tạo đơn hàng phải làm quá nhiều việc nối tiếp nhau: gọi gRPC sang product-service, lưu database, rồi lại publish event lên RabbitMQ. Đuôi p99 bị kéo dài cũng một phần do dính các request thăm dò (HALF-OPEN) của circuit breaker khi test. Đối với user-service, p99 khoảng 25ms là do hàm bcrypt.compare() được cấu hình 10 vòng mã hóa. Điều này gây tốn CPU nhưng hoàn toàn bình thường và cần thiết để đảm bảo tính bảo mật.

Appendix – Vị trí các file nguồn chính

File / Module	Service	Nội dung liên quan
src/grpc/circuitBreaker.js	order-service, product-service	Cài đặt CircuitBreaker và CircuitOpenError
src/grpc/productClient.js	order-service	gRPC client bọc CircuitBreaker; log timing ~dòng 60
src/grpc/orderClient.js	product-service	gRPC client cho DeleteOrdersByProduct (compensating tx)
src/services/orderService.js	order-service	createOrder() — timing gRPC, xử lý lỗi circuit breaker
src/services/productService.js	product-service	deleteProduct() — toàn bộ compensating transaction
src/events/orderPublisher.js	order-service	publishOrderCreated() → queue orders.created
src/events/productPublisher.js	product-service	publishProductCreated/Deleted() → products.*
src/messaging/rabbit.js	notification-service	RabbitMQ consumer; handleOrderCreated / ProductCreated / Deleted
src/push/sse.js	notification-service	SSE client registry: sendEvent(), broadcastEvent(), sendHeartbeat()
src/index.js	notification-service	GET /events endpoint; heartbeat setInterval(30000)
index.js	analytics-service	POST /track; Kafka producer → user-interactions topic
src/index.js	analytics-dashboard-service	Kafka consumer → aggregation in-memory; UI dashboard
index.js	api-gateway	JWT middleware; proxy routing; danh sách public routes
observability/	(shared)	Config OTEL Collector, Tempo, Mimir, Loki, Grafana